

Лабораторна робота №5.

Таймери програмного рівня та групи подій у FreeRTOS

Мета роботи. Ознайомитися з принципом роботи таймерів програмного рівня (Software Timers) та груп подій (Event Groups) у FreeRTOS, навчитися використовувати їх для організації подій і синхронізації задач у багатозадачному середовищі на прикладі Raspberry Pi Pico W.

Теоретичні відомості.

Таймер – це механізм, який дозволяє виконувати певну дію через визначений проміжок часу. У FreeRTOS реалізовані таймери програмного рівня (Software Timers), які використовують системний тактовий таймер ядра для відліку часу. Software Timer не виконуються у власному завданні, а обробляються спеціальним Timer Service Task, створюваним системою. Обробник таймера виконується в контексті сервісної задачі таймерів, тому не повинен містити тривалих операцій або блокуючих викликів.

Таймери дозволяють реалізувати періодичні або одноразові дії без створення окремих задач. Одноразовий (one-shot) – виконується лише один раз після закінчення періоду. Періодичний (auto-reload) – автоматично перезапускається після кожного спрацювання.

Основні функції FreeRTOS для роботи з таймерами:

- xTimerCreate() – створює таймер програмного рівня;
- xTimerStart() – запускає таймер;
- xTimerStop() – зупиняє таймер;
- xTimerReset() – скидає таймер і починає новий відлік;
- xTimerChangePeriod() – змінює період таймера;
- xTimerDelete() – видаляє таймер;
- xTimerIsTimerActive() – перевіряє, чи активний таймер.

Приклад створення таймера:

1	TimerHandle_t xTimer;
2	
3	xTimer = xTimerCreate("BlinkTimer", pdMS_TO_TICKS(1000), pdTRUE, (void *)0,
4	vTimerCallback);

```

5 xTimerStart(xTimer, 0);
6
7 void vTimerCallback(TimerHandle_t xTimer) {
8     printf("Таймер спрацював!\n");
9 }

```

Групи подій (Event Groups) у FreeRTOS є механізмом синхронізації, що дозволяє завданням чекати на встановлення одного або декількох бітів подій у певній групі. На відміну від семафорів, які сигналізують про одну подію, групи подій можуть відображати кілька незалежних станів, представлених як біти 32-бітного числа.

Event Group — структура даних, яка містить набір бітів (до 24 або 32, залежно від архітектури). Event bits — окремі біти в групі, кожен з яких представляє певну подію або стан.

Завдання може:

- Встановити біти подій (xEventGroupSetBits()),
- Очікувати встановлення певних бітів (xEventGroupWaitBits()),
- Очистити біти (xEventGroupClearBits()),
- Отримати поточний стан (xEventGroupGetBits()).

При виклику xEventGroupWaitBits() можна вказати параметри:

- xWaitForAllBits – якщо pdTRUE, завдання очікує встановлення всіх заданих бітів; якщо pdFALSE, достатньо будь-якого одного.
- xClearOnExit – якщо pdTRUE, біти очищуються після виходу з очікування.
- xTicksToWait – час очікування у тиках системного таймера.

Приклад використання:

```

1 EventGroupHandle_t xEventGroup;
2
3 #define EVENT_BIT_TIMER_DONE (1 << 0)
4 #define EVENT_BIT_SENSOR_OK (1 << 1)
5
6 void vTaskA(void *pvParameters) {
7     for (;;) {
8         // Очікування, доки таймер не встановить біти події
9         xEventGroupWaitBits(xEventGroup, EVENT_BIT_TIMER_DONE, pdTRUE,
10 pdFALSE, portMAX_DELAY);
11         printf("Подія від таймера отримана!\n");

```

12	}
13	}

Хід виконання роботи.

1. Створіть проєкт FreeRTOS у середовищі розробки VS Code.
2. Створіть таймер, який спрацьовує кожну секунду та виводить повідомлення у консоль.
3. Додайте виведення поточного часу системного тіку (xTaskGetTickCount).
4. Створіть групу подій.
5. У функції зворотного виклику (callback) таймера встановлюйте біт події `EVENT_BIT_TIMER_DONE`.
6. Створіть задачу, яка очікує встановлення цього біта, після чого виводить повідомлення у консоль.
7. Додайте другу задачу, яка генерує подію `EVENT_BIT_SENSOR_OK` (наприклад, при натисканні кнопки).
8. Створіть третю задачу, яка очікує одночасно встановлення двох бітів (`EVENT_BIT_TIMER_DONE | EVENT_BIT_SENSOR_OK`) перед виконанням дії.
9. Зробіть, щоб таймер вмикав/вимикав світлодіод кожну секунду, а інша задача зупиняла таймер при отриманні певної події (наприклад, натисканні кнопки).